

■ B.2.2 Special Input Forms

Everything you type into *Mathematica* is interpreted as an expression. Many objects that are fundamental to *Mathematica*, or in common usage, can however be input in special forms. Often you will only ever see the special forms of these objects. They are nevertheless all expressions. The table in this section gives the correspondence of special forms with standard *Mathematica* expressions.

Some special input forms, such as $a + b$ can be thought of as “operators”. Other input forms, such as $a : b$ are not so naturally considered as operators. All of them, however, are ultimately just short forms for standard *Mathematica* expressions.

<i>input form</i>	<i>full form</i>	<i>grouping</i>
$digits.digits$	(approximate number)	
n^{digits}	(base n integer)	
$n^{digits}.digits$	(base n approximate number)	
$expr:string$	<code>MessageName[expr, "string"]</code>	
forms containing #	(see below)	
forms containing %	(see below)	
forms containing _	(see below)	
$expr_1?expr_2$	<code>PatternTest[expr₁, expr₂]</code>	
$expr_1[expr_2, \dots]$	<code>expr₁[expr₂, ...]</code>	$(e[e])[e]$
$expr_1[[expr_2, \dots]]$	<code>Part[expr₁, expr₂, ...]</code>	$(e[[e]])[[e]]$
$expr++$	<code>Increment[expr]</code>	
$expr--$	<code>Decrement[expr]</code>	
$++expr$	<code>PreIncrement[expr]</code>	
$--expr$	<code>PreDecrement[expr]</code>	
$expr_1 @ expr_2$	<code>expr₁[expr₂]</code>	$e @ (e @ e)$
$expr_1 \sim expr_2 \sim expr_3$	<code>expr₂[expr₁, expr₃]</code>	$(e \sim e \sim e) \sim e \sim e$
$expr_1 /@ expr_2$	<code>Map[expr₁, expr₂]</code>	$e /@ (e /@ e)$
$expr_1 //@ expr_2$	<code>MapAll[expr₁, expr₂]</code>	$e //@ (e //@ e)$
$expr_1 @@ expr_2$	<code>Apply[expr₁, expr₂]</code>	$e @@ (e @@ e)$
$expr!$	<code>Factorial[expr]</code>	
$expr!!$	<code>Factorial2[expr]</code>	
$expr'$	<code>Derivative[1][expr]</code>	
$expr_1 ** expr_2 ** expr_3$	<code>NonCommutativeMultiply[expr₁, expr₂, expr₃]</code>	$e ** e ** e$
$expr_1 . expr_2 . expr_3$	<code>Dot[expr₁, expr₂, expr₃]</code>	$e . e . e$
$expr_1^{expr_2}$	<code>Power[expr₁, expr₂]</code>	$e^{(e^e)}$
$-expr$	<code>Times[-1, expr]</code>	
$+expr$	<code>expr</code>	
$expr_1/expr_2$	<code>expr₁ (expr₂)⁻¹</code>	$(e / e) / e$
$expr_1 expr_2 expr_3$	<code>Times[expr₁, expr₂, expr₃]</code>	$e e e$
$expr_1 * expr_2 * expr_3$	<code>Times[expr₁, expr₂, expr₃]</code>	$e * e * e$
$expr_1 + expr_2 + expr_3$	<code>Plus[expr₁, expr₂, expr₃]</code>	$e + e + e$
$expr_1 - expr_2$	<code>Subtract[expr₁, expr₂]</code>	$(e - e) - e$

Special input forms, part one.

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

<i>input form</i>	<i>full form</i>	<i>grouping</i>
$expr_1 == expr_2$	<code>Equal[$expr_1$,$expr_2$]</code>	$e == e == e$
$expr_1 != expr_2$	<code>Unequal[$expr_1$,$expr_2$]</code>	$e != e != e$
$expr_1 > expr_2$	<code>Greater[$expr_1$,$expr_2$]</code>	$e > e > e$
$expr_1 >= expr_2$	<code>GreaterEqual[$expr_1$,$expr_2$]</code>	$e >= e >= e$
$expr_1 < expr_2$	<code>Less[$expr_1$,$expr_2$]</code>	$e < e < e$
$expr_1 <= expr_2$	<code>LessEqual[$expr_1$,$expr_2$]</code>	$e <= e <= e$
<code>!expr</code>	<code>Not[expr]</code>	<code>!(!e)</code>
$expr_1 \&\& expr_2 \&\& expr_3$	<code>And[$expr_1$,$expr_2$,$expr_3$]</code>	$e \&\& e \&\& e$
$expr_1 expr_2 expr_3$	<code>Or[$expr_1$,$expr_2$,$expr_3$]</code>	$e e e$
$expr..$	<code>Repeated[expr]</code>	
$expr...$	<code>RepeatedNull[expr]</code>	
$symb: expr$	<code>Pattern[symb,expr]</code>	
$expr_1 /; expr_2$	<code>Condition[$expr_1$,$expr_2$]</code>	$(e /; e) /; e$
$expr_1 \rightarrow expr_2$	<code>Rule[$expr_1$,$expr_2$]</code>	$e \rightarrow (e \rightarrow e)$
$expr_1 \rightarrow expr_2$	<code>RuleDelayed[$expr_1$,$expr_2$]</code>	$e \rightarrow (e \rightarrow e)$
$expr_1 /. expr_2$	<code>ReplaceAll[$expr_1$,$expr_2$]</code>	$(e /. e) /. e$
$expr_1 //. expr_2$	<code>ReplaceRepeated[$expr_1$,$expr_2$]</code>	$(e //. e) //. e$
$expr_1 += expr_2$	<code>AddTo[$expr_1$,$expr_2$]</code>	
$expr_1 -= expr_2$	<code>SubtractFrom[$expr_1$,$expr_2$]</code>	
$expr_1 *= expr_2$	<code>TimesBy[$expr_1$,$expr_2$]</code>	
$expr_1 /= expr_2$	<code>DivideBy[$expr_1$,$expr_2$]</code>	
$expr \&$	<code>Function[expr]</code>	
$expr_1 // expr_2$	<code>$expr_2[expr_1]$</code>	$(e // e) // e$
$expr_1 = expr_2$	<code>Set[$expr_1$,$expr_2$]</code>	$e = (e = e)$
$expr_1 := expr_2$	<code>SetDelayed[$expr_1$,$expr_2$]</code>	
$expr_1 ^= expr_2$	<code>UpSet[$expr_1$,$expr_2$]</code>	
$expr_1 ^:= expr_2$	<code>UpSetDelayed[$expr_1$,$expr_2$]</code>	
$symb/: expr_1 = expr_2$	<code>TagSet[symb,$expr_1$,$expr_2$]</code>	
$symb/: expr_1 := expr_2$	<code>TagSetDelayed[symb,$expr_1$,$expr_2$]</code>	
$expr =.$	<code>UnSet[expr]</code>	
$symb/: expr =.$	<code>TagUnSet[symb,expr]</code>	
$expr ::= symb$	<code>Alias[expr, symb]</code>	
$expr ::=.$	<code>UnAlias[expr]</code>	

Special input forms, part two.

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

<code>expr >> filename</code>	<code>Put[expr,"filename"]</code>
<code>expr >>> filename</code>	<code>PutAppend[expr,"filename"]</code>
<code>expr₁;expr₂;expr₃</code>	<code>CompoundExpression[expr₁,expr₂,expr₃]</code>
<code>expr₁;expr₂;</code>	<code>CompoundExpression[expr₁,expr₂,Null]</code>

Special input forms, part three.

<i>input form</i>	<i>full form</i>
<code>#</code>	<code>Slot[1]</code>
<code>#n</code>	<code>Slot[n]</code>
<code>##</code>	<code>SlotSequence[1]</code>
<code>##n</code>	<code>SlotSequence[n]</code>
<code>%</code>	<code>Out[]</code>
<code>%%</code>	<code>Out[-2]</code>
<code>%%...% (n times)</code>	<code>Out[-n]</code>
<code>%n</code>	<code>Out[n]</code>
<code>_</code>	<code>Blank[]</code>
<code>_expr</code>	<code>Blank[expr]</code>
<code>_</code>	<code>BlankSequence[]</code>
<code>_expr</code>	<code>BlankSequence[expr]</code>
<code>___</code>	<code>BlankNullSequence[]</code>
<code>___expr</code>	<code>BlankNullSequence[expr]</code>
<code>..</code>	<code>Optional[Blank[]]</code>
<code>symb_</code>	<code>Pattern[symb,Blank[]]</code>
<code>symb_expr</code>	<code>Pattern[symb,Blank[expr]]</code>
<code>symb_</code>	<code>Pattern[symb,BlankSequence[]]</code>
<code>symb_expr</code>	<code>Pattern[symb,BlankSequence[expr]]</code>
<code>symb___</code>	<code>Pattern[symb,BlankNullSequence[]]</code>
<code>symb___expr</code>	<code>Pattern[symb,BlankNullSequence[expr]]</code>
<code>symb_.</code>	<code>Optional[Pattern[symb,Blank[]]]</code>

Additional special input forms.

<i>expr</i> and <i>expr</i> ;	any expression
<i>symb</i>	any symbol
<i>n</i>	a non-negative integer
<i>string</i>	"cccc" or a sequence of letters and digits
<i>filename</i>	like <i>string</i> , but can also include any of the characters !, ., /, ~, _ and \$
<i>digits</i>	a sequence of digits (including letters when <i>base</i> is above 10)

Objects used in the tables of special input forms.

Precedence and the Ordering of Input Forms

The tables of input forms are arranged in decreasing order of *precedence*. Input forms in the same box have the same precedence. Precedence determines how *Mathematica* groups terms in input expressions.

When you type $2 + 5 * 6$, *Mathematica* must decide whether this means $(2 + 5) * 6$ or $2 + (5 * 6)$. Following the standard mathematical convention, *Mathematica* chooses the second form. This choice follows from the fact that *Mathematica* assigns $*$ a “higher precedence” than $+$. The operator $*$ effectively binds its operands more tightly than $+$ does.

The general rule is that if $\otimes$ has higher precedence than $\oplus$, then $a \oplus b \otimes c$ is interpreted as $a \oplus (b \otimes c)$, and $a \otimes b \oplus c$ is interpreted as $(a \otimes b) \oplus c$.

You can always control the grouping of terms in input expressions by explicitly inserting parentheses. The precedences of different operators in *Mathematica* have been chosen so that you have to type as few parentheses as possible in typical applications. In some cases, you will still have to put in parentheses to make clear what you mean. For example, you have to distinguish $a = (b; c = d)$ from $a = b; c = d$, which means $(a = b); (c = d)$.

Grouping of Input Forms

The third columns in the tables show how multiple occurrences of a single input form, or of several input forms with the same precedence, are grouped. For example, $a/b/c$ is grouped as $(a/b)/c$ (“left associative”), while a^b^c is grouped as $a^{(b^c)}$ (“right associative”). No grouping is needed in an expression like $a + b + c$, since

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

Plus can take any number of arguments.

Where the third column of the table is left blank, the input forms cannot be grouped: you always have to insert explicit parentheses.

Spaces and Multiplication

Spaces in *Mathematica* denote multiplication, just as they do in standard mathematical notation. In addition, complete expressions that are adjacent, not necessarily separated by spaces, are also multiplied together.

```
x y z  $\longrightarrow$  x*y*z
2x  $\longrightarrow$  2*x
2(x+1)  $\longrightarrow$  2*(x+1)
c(x+1)  $\longrightarrow$  c*(x+1)
(x+1)(y+2)  $\longrightarrow$  (x+1)*(y+2)
x! y  $\longrightarrow$  x!*y
x!y  $\longrightarrow$  x!*y
```

Alternative forms for multiplication.

An expression like **x!y** could potentially mean either **(x!)*y** or **x*(!y)**. The first interpretation is chosen because **Factorial** has higher precedence than **Not**.

Spaces within single input forms are ignored. Thus, for example, **a + b** is equivalent to **a+b**. You will often want to insert spaces around lower precedence operators to improve readability.

You can give a “coefficient” for a symbol by preceding it with any sequence of digits. When you use numbers in bases larger than 10, the digits can include letters. (In bases other than 10, there must be a space between the end of the coefficient, and the beginning of the symbol name.)

x^2y , like $x^2 y$, means $(x^2) y$

$x/2y$, like $x/2 y$, means $(x/2) y$

xy is a single symbol, not $x*y$

Some cases to be careful about.

Spaces to Avoid

You should avoid inserting any spaces between the different characters in composite operators such as $/$, $=$, and $>=$. Although in some cases such spaces are allowed, they are liable to lead to confusion.

Another case where spaces must be avoided is between the characters of the pattern object $x_$. If you type $x _$, *Mathematica* will interpret this as $x*_$, rather than the single named pattern object $x_$.

Similarly, you should not insert any spaces inside pattern objects like $x: value$.

Relational Operators

Relational operators can be mixed. An expression like $a > b >= c$ is converted to `Inequality[a, Greater, b, GreaterEqual, c]`, which effectively evaluates as $(a > b) \ \&\& \ (b >= c)$. (The reason for the intermediate `Inequality` form is that it prevents objects from being evaluated twice when something like $a > b >= c$ is processed.)